2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW) | 978-1-6654-9628-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICSTW55395.2022.00057

Identifying Randomness related Flaky Tests through Divergence and Execution Tracing

Azeem Ahmad, Erik Norrestam Held, Ola Leifler, and Kristian Sandahl
Department of Computer Science
Linköping University
Linköping, Sweden
firstname.secondname@liu.se

***Abstract*—Developers often spend time to determine whether test case failures are real failures or flaky. The flaky tests, known as non-deterministic tests, change their outcomes without any changes in the codebase, thus reducing the trust of developers during a software release as well as in the quality of a product. While rerunning test cases is a common approach, it is resource intensive, unreliable, and does not uncover the actual cause of test flakiness. Our paper evaluates an approach to identify randomness-related flaky. This paper used a divergence algorithm and execution tracing techniques to identify flaky tests, which resulted in the FLAKYPY prototype. In addition, this paper discusses the cases where FLAKYPY successfully identified the flaky test as well as those cases where FLAKYPY failed. The papers discuss how the reporting mechanism of FLAKYPY can help developers in identifying the root cause of randomness-related test flakiness. Thirty-two open-source projects were used in this. We concluded that FLAKYPY can detect most of the randomness-related test flakiness. In addition, the reporting mechanism of FLAKYPY reveals sufficient information about possible root causes of test flakiness.**

***Index Terms*—flaky tests, non-deterministic tests, flaky test detection, automated root-cause analysis, execution tracing, divergence**

I. INTRODUCTION

Developers anticipate test failures to be linked to code changes when test cases fail, yet some of these test failures are unrelated to code changes. Developers spend hours examining changes in an attempt to identify the reason of a test failure, only to discover that the problem is due to test flakiness. When no modifications are made to a codebase, a flaky test will produce both passing and failing results [1], [2]. Due to the challenges that flaky tests introduce to software testing and maintenance, practitioners and researchers are paying close attention to them [3]–[8]. Re-running test cases is the most typical method for detecting flaky tests. However, this is time-consuming (Google uses 2-16% of its testing budget for rerunning tests [9]) and unreliable [4]. Determining the number of re-runs required to detect a variation in test results is challenging [4]. Much effort [1], [2], [10], [15], [17], [24] have been dedicated to flaky test detection as compared to locating root causes of test flakiness. Developers can identify test flakiness, for example, by re-executing the tests and observing the results. However, practitioners' main interest is to know what test inputs or configurations lead to test flakiness, thus requiring great attention to a frequently asked question by testers: *"why is this test case flaky?"*.

Researchers [10]–[13] have identified many root causes of test flakiness such as test order dependency, async wait, I/O, randomness, concurrency issue. However, very few studies have been conducted to investigate specific categories of test flakiness to understand deeply the root causes. To address this, the applicability of execution tracing techniques for providing root cause analysis of flaky tests in open-source Python projects belonging to the `randomness` category is investigated in this study. To limit the scope of this paper, we considered tests which fail due to the use of random numbers. Examples include not accounting for all possible values that may be generated, or generating random numbers without a seed. Many studies have found randomness to be a prevalent category and require an investigation in this direction [10]–[14]. The contributions of this paper are:

- An Analysis on reproducing randomness-related test flakiness (R1)
- A detailed analysis on the root causes of randomness-related test flakiness and how to detect it (RQ2)
- An implementation and evaluation of prototype to help testers to detect randomness-related test flakiness (RQ3)
- A detailed analysis on the root causes of those flaky tests that were not detected by the prototype (RQ4)

We investigate the following research questions:

RQ1: To what extent can we reproduce the randomness-related flakiness?
RQ2: What are the most common traits in randomness-related test flakiness?
RQ3: To what extent does FLAKYPY helps developers in identifying randomness-related test flakiness?
RQ4: When does FLAKYPY fail to detect randomness-related test flakiness?

Section II presents the related work and method about this study is presented in section III. The detailed information about the used techniques and how FLAKYPY works is presented in section III-B and IV, respectively. Section V, VII and VIII present the results, discussion and validity threats,

978-1-6654-9628-5/22/$31.00 ©2022 IEEE
DOI 10.1109/ICSTW55395.2022.00057

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

respectively.

II. Related Work

Research on flaky tests is mainly divided into causes and classification of flakiness [10], [13], [15], [16], detection of test flakiness [1], [17]–[19], and other relevant factors that affect test flakiness [11], [12], [20].

By analyzing 52 open-source Java projects, Luo [10] classified flaky tests. They inspected 201 commits of selected projects. Asynchronous wait (45%), concurrency (20%), and test order dependency (12%) were found to be the most common causes of test flakiness. Eck et al. [13] looked at the developers' perspectives of test flakiness, including the causes, efforts to solve it, importance, and problems. They worked with Mozilla developers to categorize 200 real-world flaky tests based on the nature of the flakiness, the source of the flakiness (test or production code), and the efforts to resolve the flakiness. They discovered four new types of causes such as 1) Too restrictive range (i.e., valid output values are outside the assertion range), 2) Test case timeout, 3) Platform dependency and 4) Test suit timeout.

King et al. [19] proposed a machine learning approach that uses a Bayesian network model to predict if a test is flaky or not. Lam et al. [1] developed a framework (iDFlakies) to detect and classify certain kinds of flaky tests. The framework detected flaky tests and classified them as either order-dependent (OD) or non-order-dependent (NOD) (NOD). Their tool, however, detects flaky tests by reversing the original order of the tests and rerunning them, which means that the time consumption is much higher than that of tools that do not require rerunning tests. Bertolino et al. [18] used a static approach, named FLAST, for flakiness detection using test code similarity. Different factors that affect test flakiness were investigated by Ahmad et al. [11] using a multiple case study of 5 different companies. They listed 19 unique factors that either increase, decrease or affect the ability to find test flakiness. As mentioned, there are many approaches to identify test flakiness. They mainly differ in which factors are mainly used or which frameworks/languages they support. Differential coverage is a technique for detecting flakiness proposed by Bell et al. in DeFlaker [17].

III. Methods

We present the details of data-set, experiment settings and other details in this section..

A. Obtaining the data-set and configurations

To facilitate this, a dataset (i.e., Table III-A) of locally reproducible flaky tests was compiled, based on the dataset provided by Gruber et al. [14]. This databases consists of classified flaky tests that are compatible with pytest [1]. Each project in the dataset was cloned from Github, and a specific version in which flaky behavior had been observed was checked out using a supplied hash. The projects were allocated in their own virtual environment in which dependencies could be installed by using pipenv [2]. However, some commits in the dataset were several years old, and could not be run due to dependency issues. A few of the projects also struggled to co-function with pipenv, and were run without a virtual environment as a last resort. To run the projects and attempt to reproduce flakiness, a small pytest plugin was built. This plugin allowed individual tests to be run repeatedly in isolation using the pytest CLI. To reproduce flakiness as consistently as possible, each test was initially run 100 times each.

B. The Techniques: Divergence Algorithm & Execution Tracing

The divergence algorithm employs execution tracing to pinpoint potential locations of root causes of flakiness in the code [3]. Slicing [21] was an alternate approach in this paper. However, it was skipped due to several reasons: A dynamic slice for a cohesive test may be prohibitively large, which diminishes its usefulness, runtime overhead and difficult to implement, possibly unfeasible. On the other hand, the divergence algorithm have some benefits: Straight-forward to implement based on existing pseudo-code, possibilities for augmentation and extension, and found to be useful in previous study [3]. In order to improve the efficiency of the detecting randomness-related flaky tests, another technique named *'execution tracing, variable analysis or anomaly analysis'* [15] was combined with divergence algorithm. The idea was to detect faults caused by improperly generated random numbers through local variables analysis that were consistent-in-failing. These variables take a value that is only observed in failing runs. To facilitate both the divergence algorithm and the variable analysis, different stages of the pytest session had to be extended such that the execution of each test run could be instrumented and subsequently analyzed. At the end, execution tracing [15] was used to record the test execution information.

IV. The Tool: FlakyPy

The FlakyPy tool uses divergence algorithm and variable analysis (i.e., execution tracing) on the execution of test sessions to find potential root causes of test flakiness. The workflow of the tool is as follows: (1) Allow the user to specify a test case and a number of reruns to be performed, (2) Trace the execution of each rerun, (3) Record local variables of function calls during each rerun, (4) Analyze traces using the divergence algorithm, and (5) Perform an anomaly analysis 6. Generate report.

A. Re-running test cases

To recreate and analyze flakiness, FlakyPy relies on rerunning a selected test n times, where n is any positive integer supplied by the end user. Initially, the functionality was implemented in the FlakyPy plugin itself, but after discovering that tests written in `unittest` and `nosetest` were unable to be rerun by default, `pytest-flakefinder` was used instead.

[1] https://docs.pytest.org/en/6.2.x/

[2] https://pypi.org/project/pipenv/

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Data-set Details

PID	Project URL	Commit Hash	Test function
$P_{R,1}$	/Neuraxio/Neuraxle	20c6e5713198345b43bf6899355f1b5cf65eb02c	test_automl_sequential_wrapper
$P_{R,2}$	/holgern/beem	f3c3bd84daccac1ee4ed45c1ada0f88f29ca3f2f	test_masterpwd
$P_{R,3}$	/nmasahiro/crfmnes	5fc9625c86135fb5e8824f867a08cc410d3638bd	test_run_d80_ellipsoid
$P_{R,4}$	/mkeshav/data-genie.git	cde77b7bdb445557f78d834d72d51bb62ad9e668	test_has_columns
$P_{R,5}$	/amogorkon/fuzzylogic	58bd23aeb4b6bf39bd950bcf468b0c2f5c649e69	test_eq
$P_{R,6}$	/infosmith/helpers	72c40cf1d1a46875f2956e8827a55d4232db48cb	test_raise_if_not_type
$P_{R,7}$	/jasonhamilton/hotwing-core	f4d864b1618e928bd981540b2a446bdb74023ca6	test_load_dat_file
$P_{R,8}$	/jerodg/jgutils	6c8ed6aca37d509af8ce1b57635fad62f1c846b4	test_numeric_intermixed
$P_{R,9}$	/mindey/langsplit	d30d47bd6f73852dc716f1e79225d8d49dc39daa	test_partial_autodetect
$P_{R,10}$	/CyberZHG/mxnet-octave-conv	5bbe5aa0cc6f43f242cb4a656495df70e3198f1b	test_fit_residual
$P_{R,11}$	/lungdart/packeteer	2c6e0e8683462e17ff095a89f13a10daf29ef9e0	test_variable_string
$P_{R,12}$	/JBielan/py_ev	0a2d48235b8ff2268254c151a179a2ece40cbd37	test_equity
$P_{R,13}$	/dessimozlab/pylabeledrf	23e2c2b2c73d35b6c87dd332b45da8d31c0ea63f	test_computeLRF1
$P_{R,14}$	/fzyukio/python-matlab-functions	2860bf48f904d546af75143c531750c8857670bc	test_sub2ind
$P_{R,15}$	/kithomak/pytsmp	15d58a39e016100fb44cdcc9f6115fa7736eb2bb	test_calculate_distance_profile_constant_sequence
$P_{R,16}$	/t-sakai-kure/pywsl	79a5bf104a53cb6d3498918e435bd88ac5331543	test_cv
$P_{R,17}$	/melissawm/skprocrustes	05b6fd4fcc6cb0d7b8d1115dc562a0f8f7e227b2	test_gbb_solver_known_solution_problem_1
$P_{R,18}$	/NLeSC/spreading_dye_sampler	4282f7609959a31d1b2a4832f3ed643b15c46cb6	test_for_each_cell
$P_{R,19}$	/CyberZHG/torch-layer-normalization	89f405b60f53f85da6f03fe685c190ef394ce50c	test_fit
$P_{R,20}$	/AndrewRPorter/wiki-futures	ae6bec3aafa7c2b881b2653eae1eb528f9b63bc7	test_content_size
$P_{R,21}$	/wol4aravio/Intervallum	e314ee624f09b09913475b02470103095fc076ff	test_base
$P_{R,22}$	/cktc/ashpool	3a35638ecff85da94952cebf7f8684ed564693bc	test_differ
$P_{R,23}$	/jpetrucciani/gamble.git	d17b25899183a4c4b25fe644024ff754132ae513	test_deck_init
$P_{R,24}$	/whoopnip/py-finstmt	820407a87f03377072b545923febd1086def1bec	test_annual_change_make_forecast_and_plug
$P_{R,25}$	/Zabamund/wellpathpy	f29b14a9a1c46e77ea98b141e3cdc43674654ea2	test_resample_onto_unchanged_md
$P_{R,26}$	/matzhaugen/analytic_shrinkage	752a0f90696b1ca27a8b84ada3616cdfc98004a9	test_demean
$P_{R,27}$	/glentner/CmdKit	14ec2506c532c2e599562219c2e2daf93f5e83c8	test_app_invalid_options
$P_{R,28}$	/andim/noisyopt	625b874918ac5926c3bf05d869e914cbddfffe20	test_AveragedFunction
$P_{R,29}$	/brahle/data_partitioner	c37bca05576648c3f69f8694951bb2ae9ea9ed21	test_training_percent
$P_{R,30}$	/robotools/fontMath	7f84204033c2d0a284d02e626320ee27c3dfa5ee	test_wrapUnWrapPrecision
$P_{R,31}$	/nishaq503/CHESS	6f84e7e98d328150170141621cc095dfa80ef67e	test_contains
$P_{R,32}$	/Leechael/python-cuckoo	fa7a87cd482e8a204c94b860c3066c3c1813ff36	test_unserialize_with_reading_in_utf8

B. Gathering runtime information

To support a complete analysis of the test session, all the necessary information to run the divergence-algorithm and to perform variable analysis had to be collected during the execution of each test run. The following sections describe how this information was collected.

1) Tracing the test execution: To trace and record relevant events during test execution, Python's built-in `sys`-module was utilized. This module provides access to variables and functions that interact strongly with the interpreter, and one of these functions is the `sys.settrace(fn)`-function. The `sys.settrace(fn)`-function accepts a trace-function as input, and in turn expects this trace-function to accept three arguments: *frame*, *event*, and *arg*. The *frame* argument is the current stack frame, *event* is a string $s \in$ {`'call'`, `'line'`, `'return'`, `'exception'`, `'opcode'`}, and *arg* depends on the event type. Whenever a new local scope is entered, the trace function is invoked with *event* set to `'call'`. The trace function must then return a reference to a local trace function to be used for the new scope, or return `None` if the scope could not be traced. The general flow of `sys.settrace` is displayed in figure 1.

2) Building the execution traces: Ziftci and Cavalcanti [3] describe capturing the execution flow of a test by recording which line numbers have been executed. However, as discussed in section IV-C2, only function calls that have returned before the point of divergence are eligible for anomaly analysis. Hence, additional information about function calls and function returns was also captured to facilitate this functionality. As explained in section IV-B3, each function call is associated with a unique identifier. With this information, an execution trace is constructed as a one-dimensional vector consisting of three possible entry types:

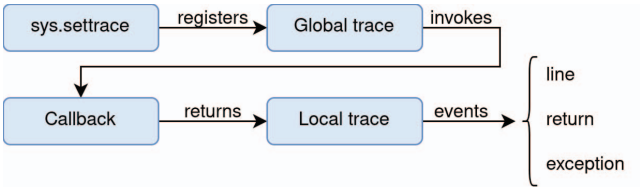


Fig. 1: The flow of the `sys.settrace`-function

1) **Line:** These entries represent an executed line number in the form of a single integer.
2) **Call:** These entries represent a function call. An entry is in the form of a triple, which is structured as `c:call_id:lineno`
3) **Return:** These entries represent a return event. An entry is in the form of a triple, which is structured as `r:call_id:call_id_of_outer_frame`

3) Capturing local variables: Since a function may be called arbitrarily many times during a test run, each function call is associated with its own unique identifier, or `call_id`. This allows local variables in the same function to be captured multiple times, if the function is called more than once. For every test run, each `call_id` is mapped to a dictionary containing the values of local variables at the point of return.

The actual mapping is done within the trace-function, which

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

is invoked with *event* set to `'return'` at the return of each function call. The local variables are then captured by accessing the stack frame through the *frame* argument.

4) The `SessionInfo` class: The recorded runtime information needs to be available during the entire test session. More general session information, such as which runs have passed and failed, must also be recorded. The `SessionInfo` class is responsible for storing information gathered across the entire session. It keeps track of which runs have passed and failed, and it maps each run number to its runtime properties, such as its execution trace, local variables, and other information regarding function calls.

5) Logging the recorded runtime properties: Writing runtime information to disk enables inter-session analysis, i.e. information from a recent session could be compared to information from previous sessions. Further, it would remove the strain of having to keep all the traced information in Python's runtime memory, something that puts FLAKYPY at risk of running out of memory when analyzing large test executions. Despite this, FLAKYPY does not write the traced information to disk by default. The main reason is that certain data types in Python are not well suited for permanent storage. To preserve the runtime properties of objects in Python, they can be *pickled*, or serialized, into a byte stream and written to a file. There are, however, data types which do not support pickling. Hence, each object that is to be pickled must first be verified as "picklable", an operation that causes tremendous overhead for large quantities of objects.

C. Analyzing runtime information

Once all test runs are complete, FLAKYPY analyses the recorded data and attempts to locate potential root causes of flakiness.

1) Divergence: The execution trace of each failing run is inspected for divergence using the DIVERGENCE algorithm. FLAKYPY does not attempt to discern whether the found divergence is relevant to the flakiness, it simply reports any divergent execution flow.

Another important usage of the divergence is to differentiate between failing runs. FLAKYPY defines a *failure group* as a set of failing runs with matching execution traces. In the anomaly analysis, each failure group is handled separately, since execution flow divergence has consequences for how the local variables are analyzed.

2) Anomalies: FLAKYPY defines an anomaly as a local variable which takes a value during a failing run that is not observed in any passing runs (with a matching execution trace). In cases where a failing run contains divergence, anomalies may only be investigated in the function calls that returned prior to the point of divergence, and only in runs with traces that match the failing run's trace up until the point of divergence.

If there are multiple failing runs with the same execution trace (i.e. no divergence between failing runs), anomalies are compared across these runs. Each anomaly is classified as belonging to one of two categories:

1) **Inconsistent-in-failing**: These anomalies have different values across failing runs. As such, they are considered to be an unlikely root cause, as their changing value did not affect the outcome of the test.
2) **Consistent-in-failing**: These anomalies have matching values across all failing runs, and are considered a likely root cause since they (1) do not appear in passing runs (2) appear in all failing runs.

FLAKYPY reports anomalies that are consistent-in-failing as a potential root cause. Anomalies which are inconsistent-in-failing are not reported as being likely root causes, but the user still has the option to investigate them.

V. RESULTS

This chapter presents the results from the pre-study, implementation and evaluation phase.

A. Reproducing Randomness flakiness - RQ1

Flaky tests reproducibility is a difficult task [11]. However, the tests that were classified as flaky due to RANDOMNESS proved to be reproducible to a much higher grade in our study. As can be seen in table II, 18 out of 26 (i.e., column 6 and 7 in table II) projects which displayed flaky behavior in the initial 100 runs (i.e., they were consistent in failing and showed divergence) were detected successfully. Looking separately, It was also noted that fewer of the flaky tests displayed a possible divergent execution flow (12/26) and captured by the tool. Only 6/22 (27%) of the investigated cases had variables that were consistent-in-failing.

B. Most Common traits in Randomness flaky tests - RQ2

When investigating Randomness flaky tests, two root causes were thought to be commonly occurring:

- Failure to consider edge/extreme values in an interval of randomly generated numbers For example, test case failing when using edge cases.
- The use of assertions that may be too restrictive. One such example is displayed in listing 1. These errors generally occurred in scientific and machine learning projects, and it should be noted that from a brief inspection, it was often unclear whether the assertion itself was the problem or if the randomly generated numbers took improper values. These assertions often use functions such as `assert_allclose` or `assertLess`.

```
def test_gbb_solver_known_solution_problem_1(self)
  :
problem = skp.ProcrustesProblem((4, 4, 2, 2),
  problemnumber=1)
mysolver = skp.GBBSolver(verbose=0)
result = mysolver.solve(problem)
assert_allclose(result.solution, problem.Xsol,
  atol=1e-3)
assert_allclose(result.fun, 1e-6, atol=1e-2)

```

Listing 1: Example of a test that fails due to a restrictive range

In terms of common libraries and keywords, 18 out of 26

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Results from attempting to recreate RANDOMNESS flakiness

Project	Runs	Failures	Possible divergence	Reproduced flakiness	Consistent-in-failing	Divergence in failures
$P_{R,1}$	100	3	No	✓	×	✓
$P_{R,2}$	100	99	Yes	✓	×	✓
$P_{R,3}$	100	2	No	✓	×	✓
$P_{R,4}$	100	79	No	×	-	-
$P_{R,5}$	100	32	No	×	-	-
$P_{R,6}$	100	10	Yes	✓	✓	✓
$P_{R,7}$	100	2	Yes	✓	×	✓
$P_{R,8}$	100	50	No	✓	✓	✓
$P_{R,9}$	100	10	No	✓	×	×
$P_{R,10}$	100	4	No	✓	×	×
$P_{R,11}$	100	1	Yes	✓	✓	×
$P_{R,12}$	100	0	N/A	✓	×	✓
$P_{R,13}$	100	6	Yes	✓	✓	×
$P_{R,14}$	100	2	Yes	✓	×	×
$P_{R,15}$	100	3	No	✓	×	×
$P_{R,16}$	100	1	No	✓	×	✓
$P_{R,17}$	100	9	Yes	✓	×	✓
$P_{R,18}$	100	2	No	×	-	-
$P_{R,19}$	100	8	No	✓	×	✓
$P_{R,20}$	200	5	Yes	✓	×	×
$P_{R,21}$	200	12	No	✓	✓	×
$P_{R,22}$	300	0	N/A	✓	✓	✓
$P_{R,23}$	300	5	Yes	✓	×	×
$P_{R,24}$	300	0	N/A	×	-	-
$P_{R,25}$	400	1	Yes	✓	×	×
$P_{R,26}$	500	6	No	✓	×	✓
$P_{R,27}$	500	377	No	-	-	-
$P_{R,28}$	500	1	Yes	-	-	-
$P_{R,29}$	600	6	No	-	-	-
$P_{R,30}$	1500	0	N/A	-	-	-
$P_{R,31}$	40000	0	N/A	-	-	-
$P_{R,32}$	50000	0	N/A	-	-	-
				Tot: 22/26	**Tot:** 6/26	**Tot:** 12/26

projects in which flakiness was recreated used Python's built-in `random` library or NumPy's `np.random`, and only one of the investigated projects did not use any external library for random number generation.

C. FLAKYPY *Helps Developers in Detecting Flakiness - RQ3*

The results of running the FLAKYPY tool a maximum of 100 times on each individual project in the dataset of flaky tests are displayed in table II. As can be seen in table II, flakiness was successfully reproduced in 22/26 tests resulting in 85% accuracy. Only 3 test cases were found to have both a divergent execution flow *and* variables that were consistent-in-failing.

The tool produces an interactive html-report, an overview of which is given in figure 2.

Each section is displayed only if such occurrences were found by FLAKYPY, i.e. if no divergence was found, or no variables were consistent-in-failing, the corresponding section would not be present in the html-report. If FLAKYPY finds variables that are consistent-in-failing, a section called "Warnings" is generated. The user may select a failure group which constitutes failing runs with matching execution flow in which consistent-in-failing variables were found. After selecting a group, a list of functions to which the variables belong is



Fig. 2: Overview of the FLAKYPY report

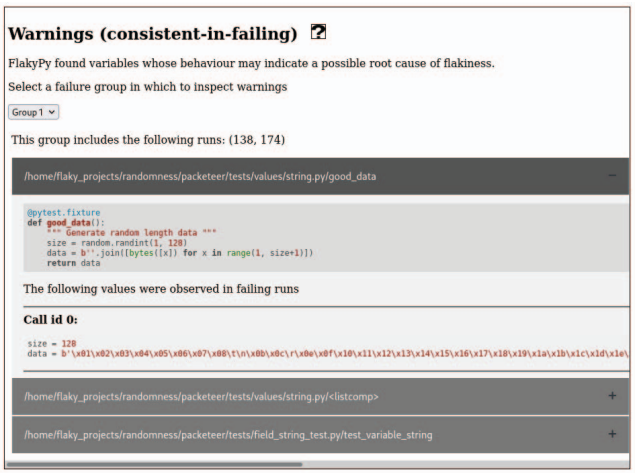


Fig. 3: Detailed view of a function reported in the "Warnings" section

shown, as displayed in figure 3. Each entry in the list is collapsible, and upon being clicked, the source code of the function and the values of variables which were consistent-in-failing are presented, as shown in figure 3 . The "Anomalies" section, which is generated if there were variables that were inconsistent-in-failing, has the exact same layout as the "Warnings" section. If FLAKYPY identified divergence in a failing run, the "Divergence report" section is generated. When the user selects a failing run to inspect, a color-coded version of the source code in which divergence occurred is presented. Each color has a different meaning:

- Gray = executed in both passing and failing runs
- Green = first divergent line executed in passing run
- Red = first divergent line executed in failing run

Alongside this, the values of local variables in the passing

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

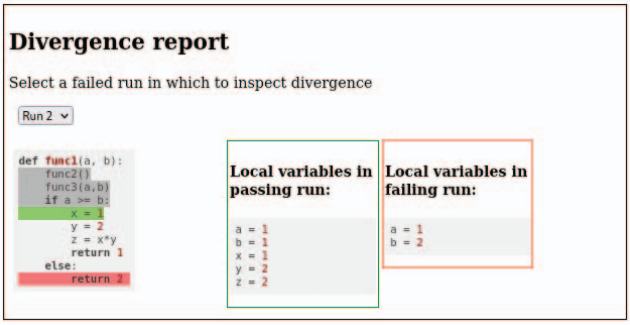


Fig. 4: Detailed view of a divergence report

and failing context are presented, as can be seen in figure 4. The chosen approach focuses on how frequently FLAKYPY can identify a potential root cause at all. This means that the possible root causes produced by FLAKYPY are not assessed or evaluated, but instead the results provide a hint of whether the implemented techniques find enough potential root causes to be considered suitable to begin with. Hence, there is the possibility that not all of the root causes found by FLAKYPY would be considered "useful" by a developer attempting to debug the test. However, the objective nature of the evaluation is positive for the reliability and validity of the results.

VI. FLAKYPY FAIL TO DETECT RANDOMNESS-RELATED TEST FLAKINESS - RQ4

This section presents the results of the source code inspection in the 5 projects, where FLAKYPY was unable to find a potential root cause. The most common characteristics found among these tests and projects are summarized below, followed by a more intricate analysis of each project.

- Randomly allocated floating point numbers are involved in $5/5$ errors.
- `numpy`-arrays are involved in $5/5$ errors.
- $5/5$ cases use non-equality assertions, e.g. `assertLess`.

A. Analysis of project $P_{R,9}$

The project is a small translation service which provides language tags for markdown. The project uses an external module called `langdetect` to detect which language is being translated, and the error in the flaky test stems from the wrong language being detected. There is no random number generation present in the test. The source of non-determinism is the usage of the `langdetect` service as displayed in listing 2, which can produce different results each time it is run if the input is too short or too ambiguous. This causes the test to fail, and because a different language is detected in each failing run, the `code` variable is not classified as consistent-in-failing. It is also noteworthy that this test case represents a very small minority in the dataset, as it does not use any random number generation.

```
def detect_language(text, max_length=2):
""" Make sure we return max_length[-letter] keys
  for languages"""
code = langdetect.detect(text)
short_code = SHORTER_CODES.get(code) if len(code)
  > max_length else code
return short_code[:max_length]

```

Listing 2: Language detection code

B. Analysis of project $P_{R,10}$

This project implements a technique aiming to reduce spatial redundancy in convolutional neural networks. There are two usages of random number generation in the test, shown in listing 3. Both generated variables are `numpy`-arrays, with `x` containing floating point numbers, and `y` containing integers. Both variables are involved in the failing assertion, `x` indirectly and `y` directly, but the machine learning elements make it difficult to determine if the random number generation is at fault, or if the training is the reason for flakiness.

```
def _test_fit(self, symbol):
logging.getLogger().setLevel(logging.DEBUG)
data_size = 4096
batch_size = 32
x = np.random.standard_normal((data_size, 3, 32,
  32))
y = np.random.randint(0, 2, data_size)
data = mx.io.NDArrayIter(x, y, batch_size=
  batch_size)
model = mx.model.FeedForward(
symbol=symbol, num_epoch=5, optimizer='adam',
numpy_batch_size=batch_size,
)
model.fit(
X=data, batch_end_callback=mx.callback.Speedometer
  (batch_size, 200),
)
predict = model.predict(x).argmax(axis=-1)
self.assertLess(np.sum(np.abs(y - predict)),
  data_size // 2)

```

Listing 3: Random number generation in $P_{R,10}$

C. Analysis of project $P_{R,15}$

This project is an implementation of the matrix profile algorithms. As displayed in listing 4, the flaky test allocates three random variables; `m` is an integer, `t` and `q` are `numpy`-arrays of floating point numbers. The FLAKYPY does not support the comparison of `numpy`-arrays, and consequently, these two variables are incompatible with the analysis that FLAKYPY performs.

D. Analysis of project $P_{R,16}$

This project implements Python codes for weakly-supervised learning. The random number generation does not take place within the test method itself, but rather in a function belonging to the SUT, as displayed in listing 5. There are several interesting aspects in this test. The lack of randomly generated variables in the test method gives rise to question whether the test itself is flaky, or if perhaps the implementation is not working as intended.

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

```
def
  test_calculate_distance_profile_constant_sequence
  (self):
n = 100
m = np.random.randint(10, n // 2)
t = np.full(n, np.random.rand())
q = np.random.rand(m)
qt = utils.sliding_dot_product(q, t)
rolling_mean, rolling_std = utils.rolling_avg_sd(t
  , m)
dp = utils.calculate_distance_profile(qt, m, np.
  mean(q), np.std(q), rolling_mean, rolling_std)
assert np.allclose(dp, np.full(n - m + 1, np.sqrt(
  m))),
"calculate_distance_profile_constant_sequence:
  distance of nonconstant " \
"query to constant sequence is sqrt(m) by
  definition."

```

Listing 4: Random number generation in $P_{R,15}$

```
def gen_twonorm_pu(n_p, n_u, prior_u, n_t):
d = 2
mu_p, mu_n = np.array([1, 1]), np.array([-1, -1])
x_p = np.random.randn(n_p, 2) + mu_p
n_up = np.random.binomial(n_u, prior_u)
n_un = n_u - n_up
x_up = np.random.randn(n_up, 2) + mu_p
x_un = np.random.randn(n_un, 2) + mu_n
x = np.r_[x_p, x_up, x_un]
y = np.r_[np.ones(n_p), np.zeros(n_u)]
x_tp = np.random.randn(n_t, 2) + mu_p
x_tn = np.random.randn(n_t, 2) + mu_n
x_t = np.r_[x_tp, x_tn]
y_t = np.r_[np.ones(n_t), -np.ones(n_t)]

return x, y, x_t, y_t

```

Listing 5: Random number generation in $P_{R,16}$

E. Analysis of project $P_{R,21}$

This project contains implementations of interval computations. The source of randomness comes from the SUT, where uniformly distributed floating point numbers are generated and placed in an array, as displayed in listing 6. This project, just as $P_{R,16}$, lacks any randomness in the test method itself.

```
def __call__(self, x: Box) -> "Interval":
def generate_point(b: Box) -> np.ndarray:
return np.random.uniform(low=b.lb, high=b.ub)
points = [generate_point(x) for _ in range(self.
  _number_of_samples)]
values = list(map(self._f, points))
return Interval(min(values), max(values))

```

Listing 6: Random number generation in $P_{R,21}$

VII. Discussion and Implications

During 2020-2021, many researchers have [2], [3], [12], [14], [18], [20], [23]–[25] have investigated the subject of test flakiness on a higher level such as listing most common root causes of test flakiness or developing a tool that can capture most of the flaky tests. Hence, test flakiness is a major problem for practitioners [11]. It is evident that researchers and practitioners should investigate each root cause of test flakiness separately and provide state-of-the-art solutions that can be used by practitioners. For example, research efforts should be directed in the specific investigation towards particular root cause of test flakiness such as randomness, I/O, concurrency, etc. The FLAKYPY tool was developed based on the analysis of randomness-related test flakiness but still it did not capture all the possible flakiness resulting in further and detailed study to list all possible root causes in the randomness category.

As discussed in listing 6, most of our efforts were dedicated towards test code to identify test flakiness. Unintentionally, we missed the possibilities of randomness-related flakiness in source code / production code. Flakiness produced by the source code requires equal attention particularly within randomness.

We observed a similar phenomena in previous studies [11], [12] that test flakiness depends on the nature of the project-s/system under test. The tests written for machine learning and artificial intelligence systems are more prone to generate test flakiness based on randomness as compared to a simple website where most of the flakiness can be produced due to testing user interfaces. Perhaps the data-set that contain more machine learning projects are well suited for this investigation due to its nature of randomness, thus motivating us to repeat a similar study with different data-set.

VIII. Validity Threats

Construction validity refers to the choice of techniques used in this study. We adopted all techniques that have been proven to have an association with flakiness in a variety of previous studies. Also, our choice of projects is limited but the projects were similar in their setup and technologies but different sizes. Future work aims to replicate the study with many more projects. To address external and internal validity, twenty six projects were used in this study for evaluation. These projects may not be representative and affect the generality of these results. The reason for the limited number of projects is because the experiment requires re-running each project 100 times locally to find flaky tests. This was a time-consuming process.

IX. Conclusion

Developers and testers are continuously annoyed with the presence of test flakiness in the test suite or product, thus dedicating many efforts to identifying root cause analysis of test flakiness. It is one thing to identify the test case that produces test flakiness and another thing to get an answer to the question: *why is this test case flaky*. Researchers and practitioners are actively working to support developers and testers to mitigate test flakiness. We took a step in this direction where we used the divergence algorithm and execution tracing techniques to identify the root causes of randomness-related test flakiness in python projects. We developed a prototype that produces sufficient reports which can help developers in locating a root cause of test flakiness through several indicators. We raised a discussion about the cases where the

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.

prototype successfully identified the test cases and where it fails and why it fails.

REFERENCES

[1] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests," in *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, Apr. 2019, pp. 312–322, iSSN: 2159-4848.

[2] W. Lam, K. Muşlu, H. Sajnani, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20. New York, NY, USA: Association for Computing Machinery, Jun. 2020, pp. 1471–1482. [Online]. Available: https://doi.org/10.1145/3377811.3381749

[3] C. Ziftci and D. Cavalcanti, "De-Flake Your Tests : Automatically Locating Root Causes of Flaky Tests in Code At Google," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Sep. 2020, pp. 736–745, iSSN: 2576-3148.

[4] D. Silva, L. Teixeira, and M. d'Amorim, "Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Sep. 2020, pp. 301–311, iSSN: 2576-3148.

[5] K. Presler-Marshall, E. Horton, S. Heckman, and K. Stolee, "Wait, Wait. No, Tell Me. Analyzing Selenium Configuration Effects on Test Flakiness," in *2019 IEEE/ACM 14th International Workshop on Automation of Software Test (AST)*, May 2019, pp. 7–13.

[6] W. Lam, S. Winter, A. Astorga, V. Stodden, and D. Marinov, "Understanding Reproducibility and Characteristics of Flaky Tests Through Test Reruns in Java Projects," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, Oct. 2020, pp. 403–413, iSSN: 2332-6549.

[7] A. Groce and J. Holmes, "Practical Automatic Lightweight Nondeterminism and Flaky Test Detection and Debugging for Python," in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security (QRS)*, Dec. 2020, pp. 188–195.

[8] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, and A. Memon, "Modeling and Ranking Flaky Tests at Apple," in *2020 IEEE/ACM 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Oct. 2020, pp. 110–119.

[9] J. Micco, "Flaky Tests at Google and How We Mitigate Them, https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html. Accessed [2019-04-15 18:48:16]." [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[10] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An Empirical Analysis of Flaky Tests," in *Proceedings of the 22Nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. New York, NY, USA: ACM, 2014, pp. 643–653, event-place: Hong Kong, China. [Online]. Available: http://doi.acm.org/10.1145/2635868.2635920

[11] A. Ahmad, O. Leifler, and K. Sandahl, "Empirical analysis of practitioners' perceptions of test flakiness factors," *Software Testing, Verification and Reliability*, vol. n/a, no. n/a, p. e1791, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.1791. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.1791

[12] ——, "An Evaluation of Machine Learning Methods for Predicting Flaky Tests," in *8th International Workshop on Quantitative Approaches to Software Quality in conjunction with the 27th Asia-Pacifc SoftwareEngineering Conference (APSEC 2020) Singapore*, Dec. 2020, pp. 37–46.

[13] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding Flaky Tests: The Developer's Perspective," *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering - ESEC/FSE 2019*, pp. 830–840, 2019, arXiv: 1907.01466. [Online]. Available: http://arxiv.org/abs/1907.01466

[14] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An Empirical Study of Flaky Tests in Python," *arXiv:2101.09077 [cs]*, Jan. 2021, arXiv: 2101.09077. [Online]. Available: http://arxiv.org/abs/2101.09077

[15] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2019. New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 101–111. [Online]. Available: https://doi.org/10.1145/3293882.3330570

[16] S. Thorve, C. Sreshtha, and N. Meng, "An Empirical Study of Flaky Tests in Android Apps," in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Sep. 2018, pp. 534–538.

[17] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically Detecting Flaky Tests," in *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, May 2018, pp. 433–444.

[18] A. Bertolino, B. Miranda, and R. Verdecchia, "Know your neighbor: fast static prediction of test flakiness," *ISTI Technical Reports*, vol. 2020, no. 001, p. 1, Jan. 2020. [Online]. Available: https://doi.org/10.32079/ISTI-TR-2020/001

[19] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a Bayesian Network Model for Predicting Flaky Automated Tests," in *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. Lisbon: IEEE Comput. Soc, Jul. 2018, pp. 100–107.

[20] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the Vocabulary of Flaky Tests?" in *Proceedings of the 17th International Conference on Mining Software Repositories*. New York, NY, USA: Association for Computing Machinery, Jun. 2020, pp. 492–502. [Online]. Available: http://doi.org/10.1145/3379597.3387482

[21] "Program Slicing - ScienceDirect." [Online]. Available: https://www-sciencedirect-com.e.bibl.liu.se/science/article/abs/pii/S0065245808606415

[22] "Analytical nonlinear shrinkage of large-dimensional covariance matrices." [Online]. Available: https://projecteuclid.org/journals/annals-of-statistics/volume-48/issue-5/Analytical-nonlinear-shrinkage-of-large-dimensional-covariance-matrices/10.1214/19-AOS1921.full

[23] Z. Dong, A. Tiwari, X. L. Yu, and A. Roychoudhury, "Concurrency-related Flaky Test Detection in Android apps," *arXiv:2005.10762 [cs]*, May 2020, arXiv: 2005.10762. [Online]. Available: http://arxiv.org/abs/2005.10762

[24] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, Jul. 2020, pp. 211–224. [Online]. Available: https://doi.org/10.1145/3395363.3397366

[25] J. Morán, C. Augusto, A. Bertolino, C. d. l. Riva, and J. Tuya, "Debugging Flaky Tests on Web Applications," Mar. 2020, pp. 454–461. [Online]. Available: https://www.scitepress.org/PublicationsDetail.aspx?ID=F80snIslQvI=&t=1

Authorized licensed use limited to: Tsinghua University. Downloaded on November 14,2022 at 12:05:42 UTC from IEEE Xplore. Restrictions apply.